Q-Learning-Guided Warm-Start LSTM Optimization for ECG Forecasting

Anonymous Author(s)
Affiliation withheld for blind review
Email: author@institution.edu

***Abstract*—Selecting the capacity and regularization of a long short-term memory (LSTM) network is a sequential, expensive design problem. This paper formulates compact LSTM configuration search as a Markov decision process and uses tabular Q-learning, driven by the Bellman update, to select the hidden width, recurrent depth, and dropout rate for one-step electrocardiogram (ECG) forecasting. The agent explores 18 configurations through seven local actions. Each selected model is trained for three epochs, and the negative validation mean squared error (MSE) is returned as reward. A warm-start mechanism reuses weights whenever consecutive configurations are structurally compatible; otherwise, training restarts. A recorded CPU experiment on 5,000 ECG5000 traces constructs 600,000 length-20 forecasting windows and evaluates 12 agent episodes. Relative to the notebook's cold-start, manual-SGD search, the Adam-based warm-start search lowers the best MSE from 0.943495 to 0.088472—a 10.66× reduction—while improving R^2 from 0.0759 to 0.9133. At the MSE-selected checkpoint, mean absolute error (MAE) falls from 0.727927 to 0.195127 (3.73×); the lowest observed MAE is 0.193173 (3.77×). These results support efficient sequential refinement, while also showing that the tenfold claim applies to MSE rather than MAE. Limitations concerning window-level splitting, stochastic repetition, and confounded ablation are stated explicitly.**

***Index Terms*—electrocardiogram, LSTM, Q-learning, Bellman equation, hyperparameter optimization, time-series forecasting, weight reuse**

I. Introduction

Electrocardiograms (ECGs) are ordered physiological signals whose local morphology and longer temporal context both matter. Public resources such as PhysioNet [1] and the UCR Time Series Archive [2] have enabled repeatable evaluation of learning algorithms on such signals. Although the ECG5000 benchmark is normally used for classification, a trace can also be converted into a self-supervised forecasting problem: a model receives the preceding samples and predicts the next amplitude. This formulation tests whether a representation captures short-range signal dynamics without using the class label as a target.

Long short-term memory (LSTM) networks [3] remain a natural recurrent baseline because their gated state mitigates vanishing gradients. However, forecast accuracy depends on hidden width, depth, regularization, optimizer, and training budget. Empirical studies of LSTM variants confirm that configuration decisions can materially change performance [4]. Exhaustive evaluation is undesirable when every candidate requires training a recurrent model.

Hyperparameter optimization (HPO) has therefore moved beyond manual grids. Random search is a strong baseline [5]; Bayesian optimization uses a surrogate of validation performance [6]; and successive-halving methods allocate resources to promising candidates [7], [8]. BOHB combines model-based selection with bandit allocation [9]. Reinforcement learning (RL) provides another view: architecture selection becomes a sequence of actions whose reward is validation quality. This idea has been demonstrated at much larger scale by neural architecture search (NAS) [10].

This work studies a deliberately small, inspectable alternative based directly on the accompanying notebook. Its contributions are threefold:

- an 18-state Markov formulation for LSTM width, depth, and dropout, navigated by a seven-action tabular Q-learning agent;
- a compatible-weight warm start that continues learning after no-op or shape-compatible transitions, combined with Adam optimization; and
- an evidence-aligned analysis of the recorded run: best MSE improves by 10.66×, while MAE improves by 3.73× at the MSE-selected checkpoint, not 10×.

The experiment is a proof of concept rather than a clinical study. In particular, it does not establish that RL alone causes the reported gain; both compared runs already use the Q-learning controller and differ in optimizer and reuse policy.

II. Related Work

A. Deep Models for Time Series

LSTM introduced an explicit memory cell and gates for preserving useful gradients over long sequences [3]. Greff *et al.* later compared major LSTM design variants and emphasized the importance of sound baselines and tuned components [4]. In time-series classification, LSTM–FCN combines recurrent and convolutional pathways [11]; broad empirical reviews show both the promise and sensitivity of deep architectures across UCR datasets [12]. Forecasting surveys similarly identify recurrent networks as competitive but stress careful preprocessing, validation, and automatic configuration [13], [14].

The present work is narrower than these general architectures: it predicts the next standardized ECG amplitude using a single LSTM followed by a linear head. Dropout is part of the search space because it can reduce co-adaptation [15]; following the underlying recurrent implementation, it is active only between stacked LSTM layers.

B. Sequential Configuration Search

Bellman's Markov decision formulation [16] provides the basis for recursively evaluating sequential choices. Q-learning learns action values without an explicit transition model and converges under standard finite-state visitation conditions [17]. NAS uses RL controllers to generate architectures [10], while ENAS reduces their cost through parameter sharing [18]. Network transformation methods likewise reuse weights when modifying architectures [19]. Inspired by these methods, the notebook keeps the controller and design space small enough for a CPU run and reuses a checkpoint only when tensor shapes are compatible. Adam [20] is used in the warm-start variant to adapt step sizes during the short evaluation budget.

The proposed search is best positioned between black-box HPO and full NAS. Random and Bayesian search normally treat a candidate configuration as an independent point. Multi-fidelity approaches make evaluation budget adaptive, while NAS controllers can generate substantially different graphs. Here, each action changes at most one coordinate of a fixed LSTM family. This locality makes transitions interpretable and a tabular value function feasible, but it also means that the agent cannot discover new cell types, skip connections, or feature extractors. The purpose is not to replace general HPO; it is to test whether a Bellman-guided local controller and safe sequential reuse can improve a constrained forecasting workflow.

III. Methodology

A. Problem Statement

Let $\mathcal{D}_{\mathrm{tr}}$ and $\mathcal{D}_{\mathrm{val}}$ be sets of input windows and next-sample targets. An LSTM configuration $\lambda = (H, L, D)$, training procedure $\mathcal{A}$, and parameter vector θ define a validation loss

$$\mathcal{L}_{\mathrm{val}}(\lambda, \theta) = \frac{1}{|\mathcal{D}_{\mathrm{val}}|} \sum_{(\mathbf{X}, y) \in \mathcal{D}_{\mathrm{val}}} \left(y - f_{\lambda,\theta}(\mathbf{X})\right)^2. \tag{1}$$

A conventional HPO objective seeks $\lambda^* = \arg\min_{\lambda \in \Lambda} \mathcal{L}_{\mathrm{val}}(\lambda, \mathcal{A}(\lambda))$. In the notebook, however, the parameters of the next candidate may inherit the immediately preceding checkpoint. Candidate quality is consequently a function of both λ and the search history. The RL formulation optimizes a finite-horizon return

$$G_k = \sum_{j=0}^{K-k-1} \gamma^j r_{k+j+1}, \qquad r_{k+1} = -\mathcal{L}_{\mathrm{val},k+1}, \tag{2}$$

for $K = 12$ episodes. The practical output is the checkpoint with minimum recorded MSE, rather than the terminal state. This distinction matters because an exploratory action near the end can lower the return while an earlier candidate remains the best deployable model.

B. Forecasting Task and LSTM

Let a standardized ECG trace be $\mathbf{x} = (x_1, \ldots, x_{140})$. For a window length $w = 20$, every valid position creates

$$\mathbf{X}_t = [x_{t-w}, \ldots, x_{t-1}], \qquad y_t = x_t. \tag{3}$$

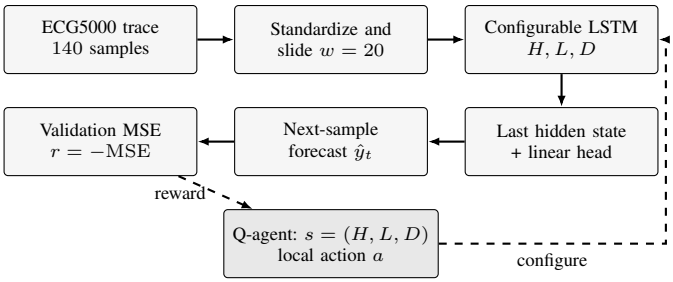

Fig. 1. Block-wise architecture of the Q-guided ECG forecaster. Solid arrows form the prediction path; dashed arrows close the search loop.

The LSTM processes $\mathbf{X}_t$ in order. For compactness, its update is

$$\begin{aligned}
\mathbf{i}_t &= \sigma(W_i x_t + U_i \mathbf{h}_{t-1} + \mathbf{b}_i), \\
\mathbf{f}_t &= \sigma(W_f x_t + U_f \mathbf{h}_{t-1} + \mathbf{b}_f), \\
\mathbf{o}_t &= \sigma(W_o x_t + U_o \mathbf{h}_{t-1} + \mathbf{b}_o), \\
\tilde{\mathbf{c}}_t &= \tanh(W_c x_t + U_c \mathbf{h}_{t-1} + \mathbf{b}_c), \\
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t),
\end{aligned} \tag{4}$$

and a linear head produces $\hat{y}_t = \mathbf{w}^\top \mathbf{h}_t + b$. Candidate quality is measured by

$$\mathrm{MSE} = \frac{1}{N} \sum_{n=1}^{N} (y_n - \hat{y}_n)^2, \quad \mathrm{MAE} = \frac{1}{N} \sum_{n=1}^{N} |y_n - \hat{y}_n|. \tag{5}$$

C. LSTM Configuration as an MDP

Each state $s = (H, L, D)$ encodes hidden width, recurrent layers, and dropout:

$$H \in \{32, 64, 128\}, \quad L \in \{1, 2\}, \quad D \in \{0, 0.2, 0.4\}. \tag{6}$$

The resulting space contains 18 states. Seven actions increment or decrement one coordinate, or keep the current state. Boundary actions are clipped, so they may become self-transitions. The deterministic transition operator is denoted $T(s, a)$. Figure 1 connects this controller to the complete forecasting path.

After training the LSTM selected by $s' = T(s, a)$, the environment returns $r = -\mathrm{MSE}_{\mathrm{val}}$. The table is updated using the one-step Bellman target:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right], \tag{7}$$

where $\alpha = 0.4$ and $\gamma = 0.8$. An ϵ-greedy policy with $\epsilon = 0.5$ balances exploration and exploitation. All Q values start at zero, the initial state is $(64, 1, 0)$, and the run lasts 12 episodes.

D. Compatible-Weight Reuse

Figure 2 summarizes one episode. A candidate model is created first. If its state-dictionary tensor shapes match the preceding checkpoint, all learned parameters are loaded before three further epochs; if loading raises a shape mismatch, the candidate starts from a fresh initialization. Thus, repeated visits to $(128, 1, 0)$ in episodes 3–6 continue training rather than discarding progress. A change in the nominal dropout value

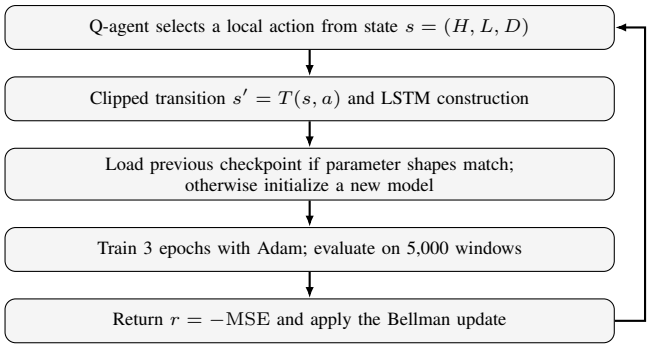

Fig. 2. One episode of the warm-start Q-learning search.

Algorithm 1 Warm-Start Q-Learning for LSTM Configuration

Require: $\mathcal{D}_{\mathrm{tr}}$, $\mathcal{D}_{\mathrm{val}}$, state set $\mathcal{S}$, actions $\mathcal{U}$
Ensure: Best checkpoint b^* and episode log $\mathcal{B}$
Set $s_0 = (64, 1, 0)$, $Q(s, a) = 0$, $\theta^- \leftarrow \varnothing$, $\mathcal{B} \leftarrow \varnothing$
for $k = 1, \ldots, 12$ **do**
  Select a_k by an ϵ-greedy policy, $\epsilon = 0.5$
  Set $s_{k+1} \leftarrow T(s_k, a_k)$ and construct its LSTM
  Load θ^- if parameter shapes match; otherwise reinitialize
  Train three epochs with Adam on $\mathcal{D}_{\mathrm{tr}}$
  Evaluate MSE, RMSE, MAE, and R^2 on $\mathcal{D}_{\mathrm{val}}$
  Set $r_k \leftarrow -\mathrm{MSE}_k$ and update Q using (7)
  Save the checkpoint; append (s_{k+1}, a_k, r_k) and metrics to $\mathcal{B}$
end for
return $b^* \leftarrow \arg\min_{b \in \mathcal{B}} \mathrm{MSE}(b)$

with one recurrent layer is also shape-compatible, although recurrent dropout is inactive when $L = 1$. Weight sharing has reduced NAS cost elsewhere [18]; here the mechanism is a simpler sequential warm start, not simultaneous supernet sharing.

IV. Experimental Protocol

The notebook downloads the UCR ECG5000 training and test files, concatenates their 5,000 traces, and separates the original class labels from 140-sample signals. A column-wise standard scaler is fitted to the full signal matrix. Equation (3) then generates 120 windows per trace, or 600,000 examples. A seeded random 80/20 window-level split forms candidate training and test pools; for CPU feasibility, the first 20,000 training and 5,000 validation examples are retained. Table I lists the remaining settings.

The comparison run uses the same agent state/action definitions and reward, but initializes each evaluated LSTM from scratch and applies a manual SGD step for three epochs. The warm-start run uses Adam and the reuse rule above. MSE, root MSE (RMSE), MAE, and R^2 are calculated over the validation windows. The notebook does not record wall-clock time, random seeds for model initialization/action selection, repeated trials, confidence intervals, or a held-out final test evaluation; none are inferred here.

In addition to (5), the recorded metrics are

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}}, \qquad R^2 = 1 - \frac{\sum_n (y_n - \hat{y}_n)^2}{\sum_n (y_n - \bar{y})^2}. \tag{8}$$

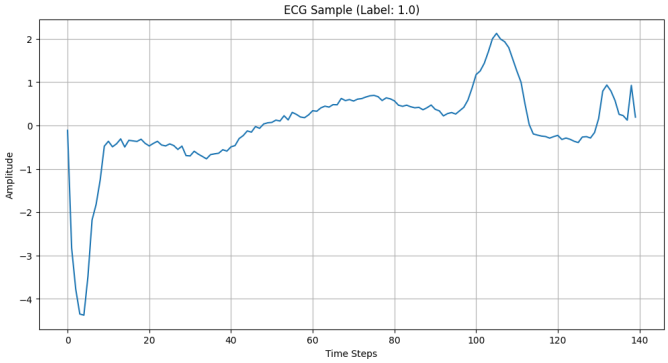

Fig. 3. An ECG5000 trace visualized in the source notebook. The class label is not used by the forecasting target.

TABLE I
Recorded Notebook Configuration

Setting	Value
Dataset / traces / length	ECG5000 / 5,000 / 140
Forecast window / target	20 / next sample
Generated / used windows	600,000 / 20,000 train, 5,000 val.
Batch size / device	512 / CPU
Episodes / epochs per episode	12 / 3
Agent $(\alpha, \gamma, \epsilon)$	$(0.4, 0.8, 0.5)$
Warm-start optimizer / rate	Adam / 10^{-3}
Baseline optimizer / rate	manual SGD / 10^{-3}
Selection objective	minimum validation MSE

MSE is the controller's optimization target because it provides a training-aligned measure and penalizes large deviations quadratically. MAE is reported alongside it because its original units and linear penalty offer a complementary view. RMSE must improve by the square root of the MSE ratio; therefore a $10.66\times$ MSE reduction mathematically corresponds to a $\sqrt{10.66} = 3.27\times$ RMSE reduction, not another tenfold change.

A. Model Size and Evaluation Budget

For input size one and a scalar linear head, a one-layer LSTM has $4H^2 + 13H + 1$ trainable parameters. A two-layer model has $12H^2 + 21H + 1$, including both recurrent layers and the head. Table II shows that the design space spans 4,513 to 199,297 parameters. Thus, the state is not merely a label: its width and depth substantially change memory and arithmetic cost. Dropout changes neither parameter count nor tensor shapes.

With 20,000 training windows and batch size 512, each epoch contains 40 mini-batches (the final one is partial), so an episode performs 120 gradient updates and the 12-episode run performs 1,440 updates across all instantiated models. The recurrent cost of a length-w window is approximately $O(wLH^2)$ when the hidden-to-hidden term dominates. Consequently, a fair large-scale extension should record elapsed time, energy, parameter count, and update budget in addition to forecast error. The current controller optimizes error only and has no incentive to prefer a smaller model when losses are equal.

TABLE II
TRAINABLE PARAMETERS BY WIDTH AND DEPTH

H	$L=1$	$L=2$	Two-/one-layer ratio
32	4,513	12,961	2.87×
64	17,217	50,497	2.93×
128	67,201	199,297	2.97×

TABLE III
SELECTED WARM-START SEARCH EPISODES

Ep.	(H, L, D)	MSE	RMSE	MAE	R^2	Reuse
1	$(32, 1, 0)$	.2031	.4506	.2720	.8011	No
3	$(128, 1, 0)$	.1350	.3674	.2471	.8678	No
4	$(128, 1, 0)$	.0996	.3156	.2021	.9024	Yes
6	$(128, 1, 0)$	**.0885**	**.2974**	.1951	**.9133**	Yes
8	$(64, 2, 0)$	.1281	.3579	.2267	.8746	No
12	$(64, 1, .2)$	.0910	.3017	**.1932**	.9109	Yes

V. RESULTS AND DISCUSSION

A. Search Trajectory

The warm-start agent first selects $(32, 1, 0)$ and obtains MSE 0.203070. It then moves through 64 and 128 hidden units. When the 128-unit architecture persists, compatible reuse allows cumulative refinement: MSE decreases from 0.134992 at episode 3 to 0.088472 at episode 6, while R^2 rises from 0.8678 to 0.9133. Later architecture changes restart training, explaining the temporary degradation in episodes 7 and 8. Selected checkpoints are reported in Table III, and the complete metric curves appear in Fig. 4.

The best-MSE model contains 67,201 parameters, only 33.7% of the largest candidate's 199,297. The recorded path therefore does not simply reward maximal capacity. Episode 8 evaluates a two-layer, 64-unit model with 50,497 parameters and produces MSE 0.128084, worse than the continuing one-layer, 128-unit model. Because the candidates receive different initialization histories, this observation is descriptive rather than a controlled width-versus-depth ablation.

B. Magnitude and Meaning of the Improvement

Table IV compares the MSE-selected checkpoints. The cold-start/manual-SGD search reaches its best MSE at $(128, 1, 0.2)$; because this model has one layer, the nominal recurrent dropout is not active. The warm-start/Adam search chooses $(128, 1, 0)$ at episode 6. Best MSE falls by 90.62%, a ratio of $0.943495/0.088472 = 10.66$. RMSE falls by 69.38% (3.27×), and MAE at the same checkpoint falls by 73.19% (3.73×). Episode 12 has the lowest MAE, 0.193173, equivalent to a 73.46% or 3.77× reduction relative to the baseline MAE.

This distinction prevents a metric-level overclaim: the notebook supports a tenfold reduction in *MSE*, not MAE. It also does not isolate the causal contribution of Q-learning. Q-learning is present in both variants; the compared pipeline additionally changes manual SGD to Adam and turns repeated compatible visits into continued training. In particular, episodes 3–6 of the 128-unit model accumulate 12 epochs, whereas every cold-start candidate receives only three.

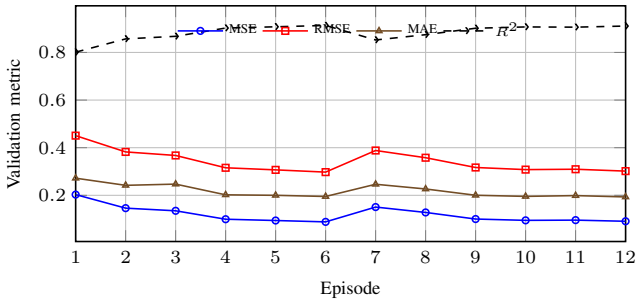

Fig. 4. Recorded validation metrics across the 12 warm-start episodes.

TABLE IV
MSE-SELECTED CHECKPOINT COMPARISON

Search variant	MSE	RMSE	MAE	R^2
Cold start + manual SGD	.943495	.971337	.727927	.0759
Warm start + Adam	**.088472**	**.297443**	**.195127**	**.9133**
Reduction	90.62%	69.38%	73.19%	+0.8374
Factor	10.66×	3.27×	3.73×	—

The observed gain therefore belongs to the complete warm-start/Adam search procedure.

The R^2 change gives another perspective. An R^2 of 0.0759 means that the cold-start checkpoint explains little of the validation variance relative to the mean predictor, whereas 0.9133 indicates much closer agreement on the recorded windows. The absolute increase is 0.8374. However, R^2 is evaluated on standardized, potentially overlapping windows; it should not be read as patient-level performance or arrhythmia-detection accuracy.

C. Practical Interpretation

The trajectory suggests two useful implementation lessons. First, a compact local action space can expose productive regions without evaluating all 18 states. Second, discarding a trained model after a no-op transition wastes the evaluation budget; compatible reuse converts revisits into additional optimization. At the same time, reuse makes rewards history-dependent: the same configuration can score differently because it has received a different cumulative number of updates. A fuller method should include training budget or checkpoint age in the state, or compare candidates under equal total compute.

The negative-MSE reward and zero initialization also create an optimistic-exploration effect. Once an action receives a negative value, unvisited actions at zero may look better to the greedy branch even before they have evidence. This can encourage coverage in a short run, but 12 episodes are far too few to visit all 18×7 state–action pairs or claim convergence. Moreover, deterministic tie breaking can privilege the first listed action. A mature implementation should randomize ties, decay ϵ, log state visitation, and separate search data from final model-selection data.

TABLE V
VALIDITY RISKS AND REQUIRED REMEDIES

Risk	Likely effect	Required remedy
Window-level split	correlated train/validation contexts	split by trace before scaling/windowing
Full-data scaling	validation information in normalization	fit scaler on training traces only
One stochastic run	unknown variance and selection stability	multiple seeds and confidence intervals
Unequal warm-start age	confounds architecture and training budget	compare equal updates or add age to state
Optimizer changed	gain cannot be assigned to reuse or RL	factorial Adam/SGD and reuse ablation
Validation-only report	optimistic post-selection estimate	untouched final test and external cohort

Warm starting is safe only at the tensor-shape level in the notebook. Matching state dictionaries allow exact loading, yet semantic changes can still matter. For example, changing dropout for a one-layer LSTM keeps identical tensors because recurrent dropout is inactive, while changing depth necessarily invalidates the checkpoint. A more general transfer rule could copy only shared submodules, initialize added layers, and record which fraction of parameters was inherited. Such partial morphisms resemble network-transformation search [19], but require careful equal-budget evaluation.

VI. VALIDITY THREATS AND FUTURE WORK

The recorded experiment has four main threats. First, scaling is fitted before splitting, and overlapping windows from a trace are randomly divided; near-duplicate temporal contexts can therefore enter both pools and inflate generalization estimates. A rigorous revision should split by entire ECG trace, fit scaling on training traces only, retain the official test split, and report a final untouched test score. Second, only the first 20,000/5,000 windows are used and only one stochastic trajectory is saved. Multiple seeded runs with mean, standard deviation, and confidence intervals are necessary.

Third, the comparison jointly changes optimizer, initialization policy, and effective epoch budget. Factorial ablations should compare fixed-configuration LSTM, cold-start Q-learning with Adam, warm-start Q-learning with SGD, equal-budget random search, Hyperband, and BOHB. Fourth, next-sample error on standardized ECG5000 traces is not a diagnostic endpoint. Evaluation should include patient-level data, clinically relevant morphology measures, inference cost, and external cohorts before biomedical conclusions are considered. Future work can also decay ϵ, use reward normalized by computation, and extend the state with learning rate and window size.

Table V converts these threats into a concrete evaluation plan. At minimum, future claims should be supported by several independently seeded searches and paired comparisons on identical trace-level splits. The final checkpoint should then be retrained under a declared budget and evaluated exactly once on an untouched test set. Reporting both the best search observation and the post-selection test result would separate optimization progress from generalization.

A. Reproducibility and Ethical Scope

The notebook records the data URLs, split seed, subset sizes, model code, controller constants, and per-episode outputs. A complete artifact should additionally pin library versions, set Python/NumPy/PyTorch random seeds, store the sampled action sequence and checkpoints, identify hardware, and export a machine-readable result table. These additions would permit exact trajectory replay and fair cost comparison.

No diagnosis, treatment recommendation, or patient-specific decision is produced. The original ECG5000 class labels are ignored, and the target is only the next standardized sample. Consequently, a low forecasting error cannot be equated with clinical sensitivity, specificity, calibration, or safety. The method should remain a signal-modeling experiment until it is evaluated on clinically defined endpoints with appropriate governance and domain review.

VII. CONCLUSION

This paper translated the supplied notebook into an explicit Bellman-guided LSTM configuration method. Across 12 recorded CPU episodes, compatible-weight reuse and Adam produced a best MSE of 0.088472 and $R^2 = 0.9133$. Against the notebook's cold-start/manual-SGD search, the MSE reduction is $10.66\times$; MAE improves by $3.73\times$ at the same checkpoint and by $3.77\times$ at its own minimum. The result motivates sequential model reuse, but the current evidence attributes the gain to the combined pipeline rather than RL alone. Trace-level splits, repeated trials, equal-budget ablations, and a held-out test are required for a definitive claim.

REFERENCES

[1] A. L. Goldberger, L. A. N. Amaral, L. Glass, J. M. Hausdorff, P. C. Ivanov, R. G. Mark, J. E. Mietus, G. B. Moody, C.-K. Peng, and H. E. Stanley, "PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000.

[2] H. A. Dau, A. Bagnall, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, and E. Keogh, "The UCR time series archive," *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 6, pp. 1293–1305, 2019.

[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink, and J. Schmidhuber, "LSTM: A search space odyssey," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 10, pp. 2222–2232, 2017.

[5] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012. [Online]. Available: https://jmlr.org/papers/v13/bergstra12a.html

[6] J. Snoek, H. Larochelle, and R. P. Adams, "Practical bayesian optimization of machine learning algorithms," in *Advances in Neural Information Processing Systems*, vol. 25, 2012, pp. 2951–2959. [Online]. Available: https://proceedings.neurips.cc/paper/2012/hash/05311655a15b75fab86956663e1819cd-Abstract.html

[7] K. Jamieson and A. Talwalkar, "Non-stochastic best arm identification and hyperparameter optimization," in *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 51, 2016, pp. 240–248. [Online]. Available: https://proceedings.mlr.press/v51/jamieson16.html

[8] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A novel bandit-based approach to hyperparameter optimization," *Journal of Machine Learning Research*, vol. 18, no. 185, pp. 1–52, 2018. [Online]. Available: https://jmlr.org/papers/v18/16-558.html

[9] S. Falkner, A. Klein, and F. Hutter, "BOHB: Robust and efficient hyperparameter optimization at scale," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 1437–1446. [Online]. Available: https://proceedings.mlr.press/v80/falkner18a.html

[10] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," in *Proceedings of the 5th International Conference on Learning Representations*, 2017. [Online]. Available: https://arxiv.org/abs/1611.01578

[11] F. Karim, S. Majumdar, H. Darabi, and S. Harford, "LSTM fully convolutional networks for time series classification," *IEEE Access*, vol. 6, pp. 1662–1669, 2018.

[12] H. I. Fawaz, G. Forestier, J. Weber, L. Idoumghar, and P.-A. Muller, "Deep learning for time series classification: A review," *Data Mining and Knowledge Discovery*, vol. 33, no. 4, pp. 917–963, 2019.

[13] H. Hewamalage, C. Bergmeir, and K. Bandara, "Recurrent neural networks for time series forecasting: Current status and future directions," *International Journal of Forecasting*, vol. 37, no. 1, pp. 388–427, 2021.

[14] B. Lim and S. Zohren, "Time-series forecasting with deep learning: A survey," *Philosophical Transactions of the Royal Society A*, vol. 379, no. 2194, p. 20200209, 2021.

[15] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014. [Online]. Available: https://jmlr.org/papers/v15/srivastava14a.html

[16] R. Bellman, "A markovian decision process," *Journal of Mathematics and Mechanics*, vol. 6, no. 5, pp. 679–684, 1957.

[17] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[18] H. Pham, M. Guan, B. Zoph, Q. Le, and J. Dean, "Efficient neural architecture search via parameter sharing," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 4095–4104. [Online]. Available: https://proceedings.mlr.press/v80/pham18a.html

[19] H. Cai, T. Chen, W. Zhang, Y. Yu, and J. Wang, "Efficient architecture search by network transformation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018, pp. 2787–2794.

[20] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proceedings of the 3rd International Conference on Learning Representations*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980